

סיכום דברים חשובים

18 בפברואר 2026

$\text{movl } a, \%$ - המעבד ניגש לכתובת של a לוקח את ערך התוצאה ושם ברגיסטר. שקול $x = a$,
 $\text{movq } \$x, \%rdi$ - המעבד לא מסתכל מה יש בתוך x , הוא לוקח את הכתובת של x ושם אותה
ב $\%$ rdi. שקול $\text{ptr} = \&x$,
 $\text{movl } (\%rdi), \%edx$ - המעבד מסתכל מה כתוב בתוך הכתובת $\%$ rdi, הולך לכתובת ומוציא
משם את הערך ושם אותו ב $\%$ edx. שקול: $\text{value} = *ptr$.

זמן ריצה נמדד לאחר שהקומפיילר כבר תרגם את הקוד לשפת מכונה
בזמן הקומפילציה, כבר מחושב כפל של שני קבועים. כלומר 2×3 מחושב בזמן קומפילציה
פקודת mov לא משנה אף דגל.
באופן עקרוני, ניתן להריץ קוד מכל סקשן שהוא - לא בהכרח מסקשן טקסט
אם גודל שורת קאש ליין הוא איקס, כתובת התחלה צריכה להתחלק ב x
 $\text{GOT\&PLT}$ - גוט היא טבלה בתוך דאטה שמאחסנת את הכתובות האמיתיות בזכרון של
פונקציות או משתנים מספריות חיצוניות. פיאלטי תפקידו לעזור ללייזי ביידינג ולקפוץ לכתובת
הראשונה בגוט או להפעיל את הלינקר בשביל למצוא אותה בפעם הראשונה. הקשר בניהם: כאשר
התוכנית קוראת לפונקציה חיצונית היא קוראת ל PLT שבתורו מסתכל ב got כדי להבין לאן לקפוץ
בזכרון - אם להביא את הפונקציה או שיש לו אותה. זה מה שמאפשר קישור דינמי בזמן ריצה.
אם $0 == (x \& (x - 1))$ אזי x הוא חזקה של 2 כי חזקה של 2 הוא מס' מהצורה
של אחד ולאחריהם כל אפסים, הקודם לחזקה של 2 הוא מס' שכולו אחדות פרט לאפס
במיקום שבו יש אחד ב x , כשנעשה and בניהם לא יתאימו בשום מיקום ויהיה אפס.
הלודר לא מעלה בזמן טעינה מידע מהסטאק ומההיפ - רק קוד, דטה (כי הם מתמלאים
בזמן ריצה)

rip מצביע לפקודה הבאה רק לאחר הפאץ' והדיקוד שעשה cpu ולפני האקסקיוט
 RIP לא תמיד מצביע לשורה הבאה - למשל jmp וכו'. הוא גם לא בהכרח מצביע
תמיד לשורה הבאה - אם היו חריגות בקוד הקוד עובר לקרנל מוד
אסור לגשת ל rip ישירות

קאש - יתרונו וחסרונו:

דיירקט מאפ: $E = 1$ לכל בלוק בזכרון יש כתובת אחת ויחידה בקאש שבה הוא יכול
להשמר, יש כאן שורה אחת בכל קבוצה
ישנן הרבה קבוצות אבל בכל אחת יש רק מקום אחד.
יתרונות: כשמחפשים נתון המעבד ניגש לסט המתאים ובודק רק שורה אחת, זה מהיר
חומרתית
חסרונות: אם שתי כתובות שונות רבות על אותו סט - הן יזרקו אחת את השניה כל
הזמן. יש כאן סכנה להתנגשויות רבות.
פולי אסוסאטיב: $S = 1$

קבוצה אחת ענקית שמכילה את כל השורות בקאש.
 יתרון: גמישות מוחלטת, נתון יכול להכנס לכל שורה פנויה, אין התנגשויות עד שהקאש מלא פיזית
 חסרונות: כמחפשים נתון, צריך להשוות את *tag* שלו מול כל השורות במקביל וזה לוקח הרבה זמן
 $E = k$ קיי אסוסאטיב: $E = k$
 מחלקים את הקאש ל-*S* קבוצות ובכל קבוצה *k* שורות
 יתרונות: צמצום משמעותי של התנגשויות, בניגוד לדיירקט מאפ כאן לכל כתובת יש קיי אפשרויות. מונע מצב ששתי כתובות רבות על אותו מקום.
 יעילות גבוהה.
 חסרונות: זמן גישה איטי יותר - בניגוד לדיירקט מאפ, צריך לבצע השוואת טאגים לקיי שורות במקביל

מה אם יש יותר או פחות שורות קאש ליין? יותר שורות: פחות *miss* כי יותר מידע נשמר, אבל זמן חיפוש יגדל. פחות שורות: התנגשויות יותר, כי צריך למחוק מידע קיים בשביל להכניס חדש אבל זה מהיר יותר לחיפוש
מה אם בסט יש רק שורה אחת - עדיין צריך *valid*? בוודאי. בלי הוואליד ביט אין דרך לדעת אם השורה כבר מעודכנת למה שיש בראמ או שלא. בלי וואליד ביט יתכן שהזכרון בראמ לא יהיה מעודכן.

:*Shell*

ממשק בין המשתמש למערכת ההפעלה
 משתני הסביבה:
PATH - מכיל רשימת נתיבים שבהם השל מחפש קבצי הרצה כשאתה מקליד פקודה
PWD - שומר נתיב של תיקייה נוכחית שבה אתה נמצא
HOME - נתיב לתיקיית בית של יוזר
SHELL - נתיב לקובץ הרצה של השל הנוכחי (*bin/bash*)
USER - שם משתמש נוכחי
printenv - מציג את משתני הסביבה
export - הופך משתנה של רגיל למשתנה סביבה, תוכניות שיופעלו מהשל יכירו אותו
echo - מדפיסה טקסט או ערכי משתנים למסך
chmod - משנה הרשאות לקובץ
_start - תפקידיה: להגדיר מחסנית ורגיסטרים, לאתחל - להריץ בנאים גלובליים, להעביר *arg, argv* למיין, לקרוא למיין - להעביר את השליטה למיין, לאחר שמיין מסתיימת מקבלת את השליטה חזרה ומבצעת סיסקאל לסגירת התהליך בצורה נקיה
strace - כלי דיבוג שמציג את הקריאות למערכת ההפעלה - כל הסיסקאלס,

אופטימיזציות:

א. לולאה הפוכה של i - עוזרת רק כאשר יש השוואה לאפס